

KNN - Code Notebook

Virtual Labs - Dayalbagh Educational Institute

Step 1: Importing Libraries

Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
print("Imported Analysis and Plotting libraries")
```

Output:

Imported Analysis and Plotting libraries

Import scikit-learn modules

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import label_binarize
from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix
)
print("Imported Sklearn Modules")
```

Output:

Imported Sklearn Modules

Step 2: Loading Dataset

Loading and Reading Data

```
iris = load_iris()
X = iris.data
y = iris.target
print("Extracted Feature and Target as numpy Arrays")
```

Output:

Extracted Feature and Target as numpy Arrays

extracting feature names

```

names = iris.feature_names

# extracting target names
target_names = iris.target_names

# making a dataframe with column names
iris_df = pd.DataFrame(X, columns=names)
iris_df["label"] = [target_names[i] for i in y]
iris_df.head()

```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Step 3: Data Analysis

Data Analysis

```

# Overview
iris_df.info()

```

Output:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)     150 non-null   float64
1   sepal width (cm)      150 non-null   float64
2   petal length (cm)     150 non-null   float64
3   petal width (cm)      150 non-null   float64
4   label                 150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

```

Target counts

```
iris_df["label"].value_counts()
```

Output:

```

label
setosa      50
versicolor  50
virginica    50
Name: count, dtype: int64

```

Dataset description

```
iris_df.describe()
```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Finding any null values

```
iris_df.isnull().sum()
```

Output:

```
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
label                0
dtype: int64
```

Plotting a histogram distribution of data points with labelled classes

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
for ax, col in zip(axes.ravel(), names):
    sns.histplot(
        data=iris_df, x=col, hue="label",
        kde=True, element="step", ax=ax
    )
    ax.set_xlabel(col, fontweight='bold')
    ax.set_ylabel("Count", fontweight='bold')
plt.suptitle("Feature Distributions by Class", y=1.02, fontweight='bold')
plt.tight_layout()
plt.show()
```

Plotting correlation between features

```
plt.figure(figsize=(10, 8))
ax = sns.heatmap(
    iris_df.iloc[:, :4].corr(),
    annot=True,
    cmap="crest",
    fmt=".2f",
    linewidths=0.7,
    # Force annotation text to be black
    annot_kws={"color": "black"}
)
plt.title("Feature Correlation Heatmap", fontweight='bold')
plt.show()
```

Step 4: Data Preprocessing

Data Preprocessing

```
# Data Encoding
# we already have encoded targets from sklearn datasets
iris_df["target"] = y
iris_df.head()
```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label	target
0	5.1	3.5	1.4	0.2	setosa	0
1	4.9	3.0	1.4	0.2	setosa	0
2	4.7	3.2	1.3	0.2	setosa	0
3	4.6	3.1	1.5	0.2	setosa	0
4	5.0	3.6	1.4	0.2	setosa	0

Stratification is important to balance classes between split

```
X = iris_df.iloc[:, :4].values
y = iris_df["target"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3,
    stratify=y,
    random_state=42
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

Output:

```
(105, 4) (45, 4) (105,) (45,)
```

Scaling the values

```
scaler = StandardScaler()
scaler
```

Output:

```
StandardScaler()
```

Apply scaling to training and test data

```
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
print("Values scaled with Standard Scaler")
```

Output:

```
Values scaled with Standard Scaler
```

View original training data (before scaling)

```
X_train[:5]
```

Output:

```
array([[5.1, 2.5, 3. , 1.1],
       [6.2, 2.2, 4.5, 1.5],
       [5.1, 3.8, 1.5, 0.3],
       [6.8, 3.2, 5.9, 2.3],
       [5.7, 2.8, 4.1, 1.3]])
```

View scaled training data (after scaling)

```
X_train_s[:5]
```

Output:

```
array([[-0.90045861, -1.22024754, -0.4419858 , -0.13661044],
       [ 0.38036614, -1.87955796,  0.40282929,  0.38029394],
       [-0.90045861,  1.63676428, -1.2868009 , -1.17041921],
       [ 1.07899781,  0.31814344,  1.19132338,  1.41410271],
       [-0.20182693, -0.56093712,  0.17754527,  0.12184175]])
```

Binarizing encoded labels for ROC-AUC metric calculation

```
y_test_bin = label_binarize(y_test, classes=[0,1,2])
y_test_bin[:5]
```

Output:

```
array([[0, 0, 1],
       [0, 1, 0],
       [0, 0, 1],
       [0, 1, 0],
       [0, 0, 1]])
```

Step 5: Model Training

Model Training

```
# k = 10 (Heuristic Rule)
model = KNeighborsClassifier(n_neighbors=10)
model.fit(X_train_s, y_train)
```

Output:

```
KNeighborsClassifier(n_neighbors=10)
```

Step 6: Model Evaluation

Model Evaluation

```
y_pred = model.predict(X_test_s)
y_pred
```

Output:

```
array([2, 1, 1, 1, 2, 2, 1, 1, 0, 2, 0, 0, 2, 2, 0, 2, 1, 0, 0, 0, 1, 0,
       1, 2, 1, 1, 1, 1, 1, 0, 2, 2, 1, 0, 2, 0, 0, 0, 0, 1, 1, 0, 1, 2,
       1])
```

Get prediction probabilities

```
y_prob = model.predict_proba(X_test_s)
y_prob[:5]
```

Output:

```
array([[0. , 0.1, 0.9],
       [0. , 0.9, 0.1],
       [0. , 0.7, 0.3],
       [0. , 0.7, 0.3],
       [0. , 0.4, 0.6]])
```

Generate classification report

```
print("\nClassification Report (k=10):")
print(classification_report(
    y_test,
    y_pred,
    target_names=target_names,
    digits=4
))
```

Output:

```
Classification Report (k=10):
precision    recall  f1-score   support

setosa       1.0000    1.0000    1.0000        15
versicolor   0.8333    1.0000    0.9091        15
virginica     1.0000    0.8000    0.8889        15
accuracy                   0.9333        45
macro avg    0.9444    0.9333    0.9327        45
weighted avg    0.9444    0.9333    0.9327        45
```

Visualize confusion matrix heatmap

```
plt.figure(figsize=(7, 5))
sns.heatmap(
    confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Greens",
    xticklabels=target_names, yticklabels=target_names
)
plt.xlabel("Predicted", fontweight='bold')
plt.ylabel("True", fontweight='bold')
plt.title("Confusion Matrix Heatmap (KNN, k=10)", fontweight='bold')
plt.tight_layout()
plt.show()
```

Step 7: Model Simulation

Final Metrics

```
# Accuracy will be same as F1 score as dataset is perfectly balanced and stratified
print(f'''Accuracy    : {accuracy_score(y_test, y_pred)}
Precision   : {precision_score(y_test, y_pred, average="macro")}
Recall      : {recall_score(y_test, y_pred, average="macro")}
F1          : {f1_score(y_test, y_pred, average="macro")}
ROC         : {roc_auc_score(y_test_bin, y_prob, multi_class="ovr", average="macro")}
''')
```

Output:

```
Accuracy    : 0.9333333333333333
Precision    : 0.9444444444444445
Recall      : 0.9333333333333332
F1          : 0.9326599326599326
ROC         : 0.9948148148148149
```

Model Training

```
# k = 10 (Heuristic Rule)
model = KNeighborsClassifier(n_neighbors=10, metric='${metric}')
model.fit(X_train_s, y_train)
print("Model trained using ${label} distance with k=10")
```

Output:

```
Model trained using ${label} distance with k=10
KNeighborsClassifier(metric='${metric}', n_neighbors=10)
```

Model Evaluation

```
y_pred = model.predict(X_test_s)
y_pred
```

Output:

```
array([2, 1, 1, 1, 2, 2, 1, 1, 0, 2, 0, 0, 2, 2, 0, 2, 1, 0, 0, 0, 1, 0,
1, 2, 1, 1, 1, 1, 1, 0, 2, 2, 1, 0, 2, 0, 0, 0, 0, 0, 1, 1, 0, 1, 2,
1])
```

Get prediction probabilities

```
y_prob = model.predict_proba(X_test_s)
y_prob[:5]
```

Output:

```
array([[0. , 0.1, 0.9],
[0. , 0.9, 0.1],
[0. , 0.7, 0.3],
[0. , 0.7, 0.3],
[0. , 0.4, 0.6]])
```

Generate classification report

```
print("\n\nClassification Report (${label}, k=10):")
print(classification_report(
    y_test,
    y_pred,
    target_names=target_names,
    digits=4
))
```

Output:

```
Classification Report (${label}, k=10):
precision    recall  f1-score   support
setosa       1.0000    1.0000    1.0000        15
versicolor   0.8333    1.0000    0.9091        15
virginica     1.0000    0.8000    0.8889        15
accuracy                   0.9333        45
macro avg    0.9444    0.9333    0.9327        45
weighted avg 0.9444    0.9333    0.9327        45
```

Visualize confusion matrix heatmap

```
plt.figure(figsize=(7, 5))
sns.heatmap(
    confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Greens",
    xticklabels=target_names, yticklabels=target_names
)
plt.xlabel("Predicted", fontweight='bold')
plt.ylabel("True", fontweight='bold')
plt.title("Confusion Matrix Heatmap (KNN, ${label}, k=10)", fontweight='bold')
plt.tight_layout()
plt.show()
```

Step 8: Model Simulation**# Final Metrics**

```
# Accuracy will be same as F1 score as dataset is perfectly balanced and stratified
print(f'''Accuracy   : {accuracy_score(y_test, y_pred)}
Precision  : {precision_score(y_test, y_pred, average="macro")}
Recall     : {recall_score(y_test, y_pred, average="macro")}
F1         : {f1_score(y_test, y_pred, average="macro")}
ROC        : {roc_auc_score(y_test_bin, y_prob, multi_class="ovr", average="macro")}
''')
```

Output:

```
Accuracy   : 0.9333333333333333
Precision  : 0.9444444444444445
Recall     : 0.9333333333333332
F1         : 0.9326599326599326
ROC        : 0.9948148148148149
```